



Ain Shams University, Cairo, Egypt
Faculty of Engineering,
International Credit Hours Engineering Programs (ICHEP)

CSE241 Object-Oriented Computer Programming

Desktop Hotel Reservation System

Milestone 2 Final Report

Submitted to: -

Dr. Mahmoud Khalil
Eng. Mazen Tarek
Eng. Mario Remon

Prepared by: -

Yassin Mohamed Yehia (25p0006)
Seif Hazem (25p0005)
Abdullah Alaa (25p0139)
Mohamed AlaaEldin (25p0202)
Karim Ismail Samy (25P0060)

Date of submission: -

7/5/2026

GitHub Repository:

<https://github.com/YassinMohamed07/ProjectOOPHotelSystem>

YouTube Demo Video:

<https://youtu.be/xBq9s3OUMR8>

Contents

1	Introduction & Objective	2
2	Team Responsibilities & Contributions	2
3	Advanced Features (Bonus Implementations)	3
3.1	Multi-threading for Background Operations	3
3.2	Basic Networking (Client-Server Chat)	4
4	GUI Design & Screens Implementation	4
4.1	Login & Registration	4
4.2	Guest Operations	4
4.3	Staff Operations (Admin & Receptionist)	6
4.4	Live Chat & Error Handling Demonstrations	7
5	Test Cases (Valid & Invalid Scenarios)	7

1 Introduction & Objective

This report documents the implementation of Milestone 2 for the Desktop Hotel Reservation System. Building upon the solid Object-Oriented backend developed in Milestone 1, the objective of this phase was to deliver a fully functional, interactive Graphical User Interface (GUI) using the JavaFX framework.

In addition to standard UI development using Model-View-Controller (MVC) architecture and FXML separation, our team successfully implemented advanced features including multi-threading for asynchronous background operations and basic client-server networking (Sockets) for real-time chat functionality between guests and hotel staff.

2 Team Responsibilities & Contributions

The workload for Milestone 2 was divided to ensure fair contribution while assigning complex architectural tasks to members.

Team Member	ID	Contributions (Milestone 2)
Yassin Mohamed Yehia	25p0006	Lead UI Architect: Handled the largest and most complex chunk of the GUI. Designed the MVC architecture, implemented complex controllers (ReceptionistDashboardController , AdminDashboardController , RoomBrowsingController), and managed the overarching CSS styling and dynamic JavaFX layout integrations.
Seif Hazem	25p0005	Bonus & Advanced Features: Designed and implemented all multi-threading logic (using JavaFX Task and Platform.runLater). Built the complete Client-Server networking architecture (ChatServer , ChatClient) for real-time live chat functionality.
Abdullah Alaa	25p0139	Moderate GUI Logic: Developed the CheckoutController and ReservationController . Implemented complex TableView data bindings, dynamic invoice generation, and the suite of JavaFX Alert dialogues for error/success handling.
Mohamed AlaaEldin	25p0202	UI Integration: Developed the GuestDashboardController . Ensured seamless data binding between the active Guest model and the profile/balance UI components.

Karim Ismail Samy	25P0060	Foundational UI & Nav: Implemented the <code>LoginRegisterController</code> . Built the <code>SceneNavigator</code> utility for seamless view switching using a shared <code>StackPane</code> , and assisted with integration testing.
--------------------------	---------	---

3 Advanced Features (Bonus Implementations)

3.1 Multi-threading for Background Operations

To ensure a smooth, non-freezing user experience, we integrated Java multithreading using JavaFX's `Task` class.

- **Background Data Loading:** In the `RoomBrowsingController` and `ReservationController`, room searches and reservation history fetching are executed on background daemon threads.
- **Thread Safety:** We strictly adhered to JavaFX thread-safety rules. Background threads never update the UI directly; instead, `Task.setOnSucceeded()` and `Platform.runLater()` are utilized to push data to the `TableViews` once the background computation is complete.

3.2 Basic Networking (Client-Server Chat)

We implemented a live chat feature allowing logged-in guests to communicate with the receptionist in real time.

- **Architecture:** A `ChatServer` class acts as the centralized hub using `ServerSocket`. It handles multiple concurrent clients by spawning a new `ClientHandler` thread for every connection.
- **Integration:** The `GuestDashboardController` and `ReceptionistDashboardController` instantiate a `ChatClient` (using Java Sockets) upon initialization. Messages are broadcasted securely across the network and appended to the JavaFX `TextAreas` dynamically.

4 GUI Design & Screens Implementation

The application strictly utilizes JavaFX (no Swing) with FXML for layout definition, CSS for professional styling (dark theme with vibrant accents), and Java Controllers for logic.

4.1 Login & Registration

Handles both Guest and Staff authentication using `InvalidCredentialException` handling. Includes robust registration validation for password strength and age verification.

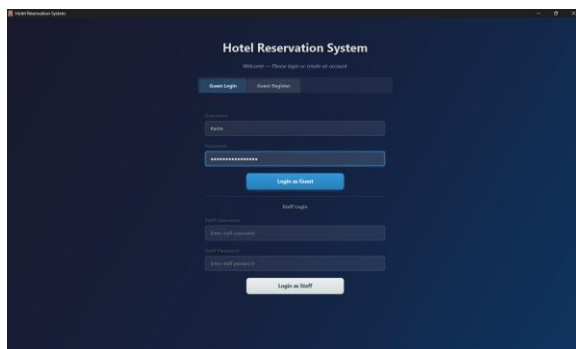


Figure 1: User Login Interface

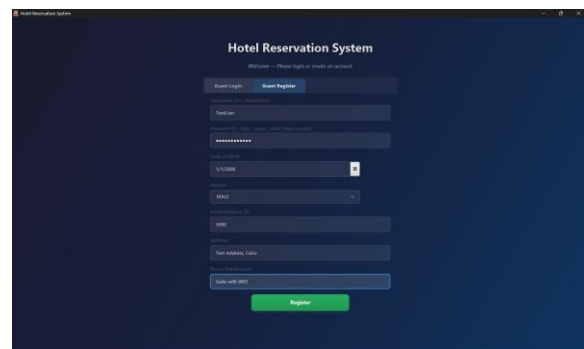


Figure 2: Guest Registration Form

4.2 Guest Operations

The Guest interface allows users to view their dashboard, browse rooms with dynamic filters (Type, Max Price, Dates), add extra paid amenities, view history, and checkout.

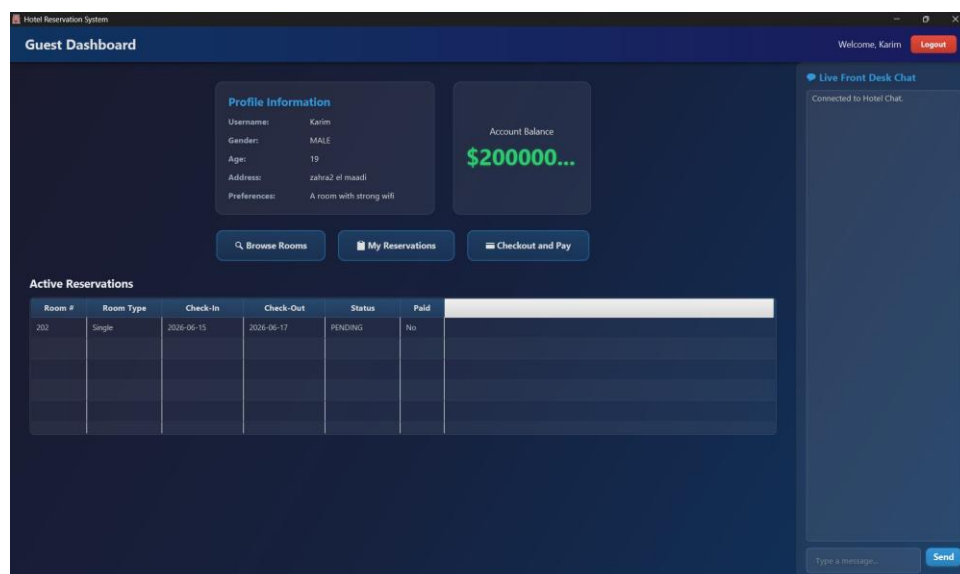


Figure 3: Guest Dashboard displaying profile info, active bookings, and balance.

Browse Rooms

Search Filters

Room Type: DOUBLE, Max Price / Night (\$): 4000, Check-In Date: 6/15/2026, Check-Out Date: 6/17/2026

Amenities: ☒ Basic Wi-Fi, ☒ Standard Smart TV, ☒ Air Conditioning & Heating, ☒ In-Room Safe, ☒ Daily Housekeeping, ☒ Work Desk & Chair, ☒ Coffee & Tea Station

Search Rooms, Clear Filters

Available Rooms 1 room(s) found

Room #	Type	Base Price	Amenities	Total / Night
102	Double	\$3500.00	Basic Wi-Fi, Standard Smart TV, Air Con...	\$3500.00

Extra Amenities (Optional)

Select a room above, then add paid extras to your booking

Extra Amenity: Spa & Sauna Access (\$75.00) Add Extra Clear Extra

Selected: Spa & Sauna Access (\$75.00/night) | Extra cost: \$75.00/night

Book Selected Room

Figure 4: Room Browsing Screen with dynamic filters and optional extra amenities selection.

My Reservations

Booking History

Room #	Room Type	Check-In	Check-Out	Status	Paid	Total Cost
102	Single	2024-06-15	2024-06-17	Pending	No	\$4000.00

Cancel Selected Reservation, Reserve Rooms

Figure 5: Reservation Management (Booking History)

Checkout & Payment

Unpaid Reservations

Room #	Type	Check-In	Check-Out	Status	Total Cost
102	Single	2024-06-15	2024-06-17	Pending	\$4000.00

Payment Details

Payment Method: Credit Card

Generate Invoice, Confirm Payment

Invoice Summary

Room: 102, Room Type: Single, Room #: 102, Check-In: 2024-06-15, Check-Out: 2024-06-17, Total Cost: \$4000.00

Invoice generated for Room 102

Figure 6: Checkout & Payment with Auto-Invoice

4.3 Staff Operations (Admin & Receptionist)

Staff dashboards utilize complex `TableView` components to manage the hotel's backend entities. The **Admin** manages Rooms, Amenities, and registers new staff. The **Receptionist** handles Check-ins, Check-outs, and payment processing.

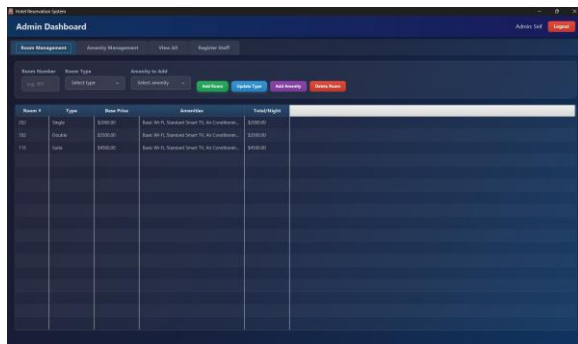


Figure 7: Admin: Room Management

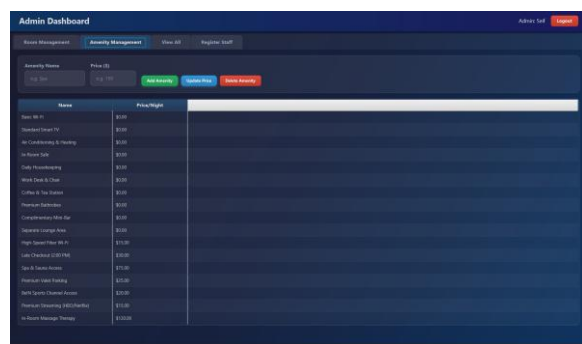


Figure 8: Admin: Amenity Management

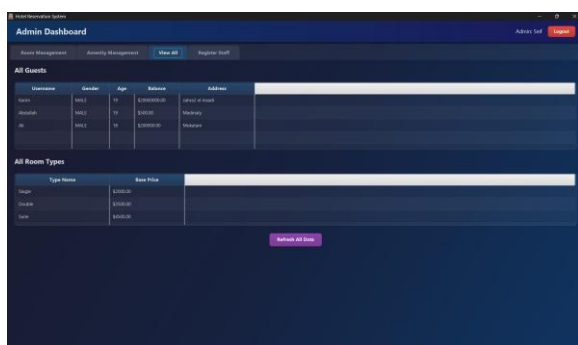


Figure 9: Admin: View All System Data

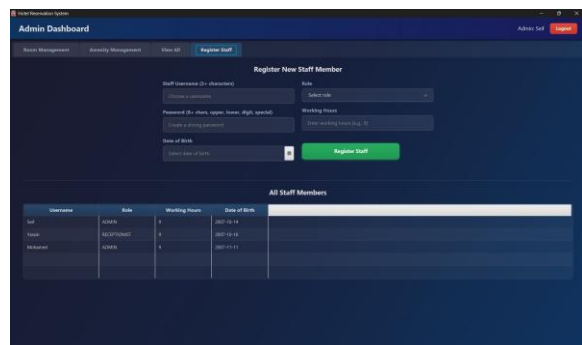


Figure 10: Admin: Staff Registration

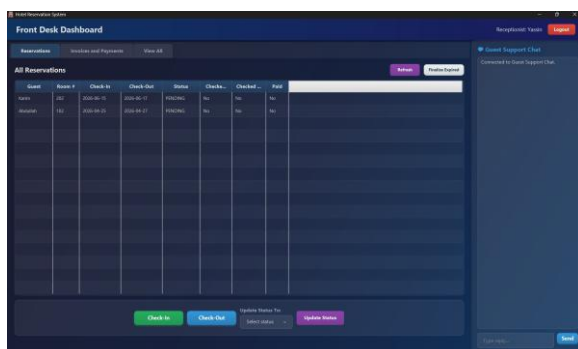


Figure 11: Receptionist: Managing Reservations

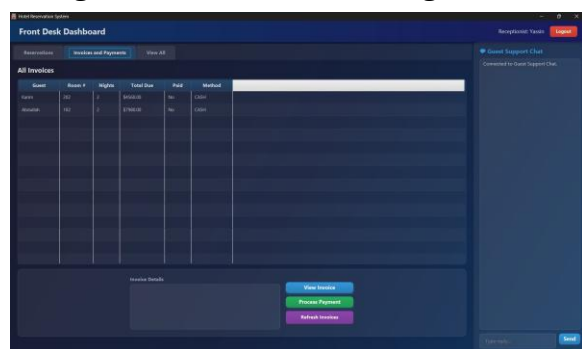


Figure 12: Receptionist: Invoice & Payments

4.4 Live Chat & Error Handling Demonstrations

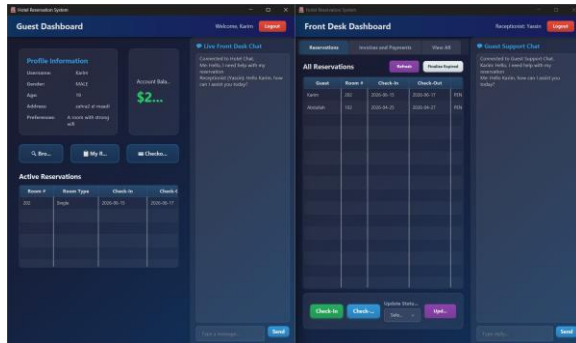


Figure 13: Live Socket Chat between Guest and Receptionist

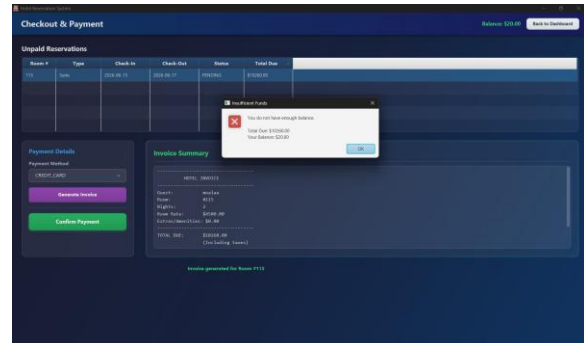


Figure 14: JavaFX Alert: Insufficient Funds Validation

5 Test Cases (Valid & Invalid Scenarios)

Extensive testing was conducted linking the UI elements to the custom Backend Exceptions ([InvalidCredentialException](#), [WeakPwordException](#), [InvalidDateException](#)).

Feature	Scenario (Input)	Expected Output / Behavior	Status
Registration	Valid data, strong password (Pass@123), Age 19.	Success alert; user added to database.	Pass
Registration	Weak password (password123 - no special char).	Red error label: <i>"Password must contain at least one special character"</i> .	Pass
Registration	Age under 18 (e.g., DOB is 2015).	Red error label: <i>"Must be 18+ to register"</i> .	Pass
Login	Incorrect username or password.	Red error label: <i>"Wrong password"</i> or <i>"User not found"</i> .	Pass
Booking	Check-in date is after Check-out date.	Warning Alert: <i>"Check-in must be before check-out"</i> .	Pass
Booking	Dates conflict with existing reservation.	Room does not appear in Table-View results.	Pass
Checkout	User tries to pay invoice but balance is lower than total due.	Error Alert: <i>"You do not have enough balance."</i> (See Fig. 14)	Pass
Admin	Adds an amenity that already exists.	Status Label (Red): <i>"Amenity already exists!"</i>	Pass
Receptionist	Tries to check-in a user before their actual check-in date.	Status Label (Red): <i>"Cannot check in. Reservation starts on [Date]"</i>	Pass